# NFC Student ID Card Security Research

College of Charleston — Blackboard Transact System

*Passive Enumeration & Architecture Analysis | May 2026*

## 1. What Is This Research?

This document explains how a College of Charleston student ID card works at a technical level. All information was obtained using passive, unauthenticated methods — meaning we only read data the card broadcasts freely without needing any passwords or secret keys.

Think of it like reading the label on a locked safe. We can see who made it, how many compartments it has, and what kind of lock is on each one — but we cannot open it without the combination.

## 2. The Card Hardware

Your CofC student ID uses a chip called a Mifare DESFire EV1, made by NXP Semiconductors. It communicates wirelessly using the ISO 14443-A standard — the same technology behind contactless credit cards and phone payments.

| Property | Value | What It Means |
|---|---|---|
| Card Type | Mifare DESFire EV1 | A widely used smart card chip |
| Chip Manufacturer | NXP Semiconductors | Industry standard NFC chip maker |
| Hardware Version | EV1, v1.1 | First generation DESFire with AES support |
| Storage Size | 8 kilobytes | Total memory available on the chip |
| Free Memory | 4,192 bytes | Storage not currently in use |
| Communication Standard | ISO 14443-4A | The wireless protocol it speaks |
| UID (Unique ID) | 04 45 06 2A 08 70 80 | Globally unique 7-byte chip identifier |

The UID is like a serial number burned into the chip at the factory. It cannot be changed and is broadcast to any reader that asks — no authentication required. This is what a reader logs when you tap your card.

## 3. How the Card Is Organized

A DESFire card is structured like a filing cabinet. At the top level sits the PICC (Proximity Integrated Circuit Card) — the card's master controller. Inside it, separate Applications act like individual locked drawers, each serving a different campus function.

**Your card has three applications:**

| Application ID | Role | Files | Access Model |
|---|---|---|---|
| BBBBBB | Campus Identity (Student ID) | 2 files, 48 bytes each | Single admin key — fully locked |
| B3BBBB | Access Control (Doors/Buildings) | 1 file, 48 bytes | Split key — readers get read-only |
| DABBBB | Transactional (Dining/Printing) | 1 file, 48 bytes | Split key — terminals get read-only |

## 4. How Keys and Security Work

Each application is protected by cryptographic keys — long strings of random data that act as passwords. Your card uses AES-128 encryption, the same standard used by banks and the US government.

**A critical point: the keys are NOT stored on the card itself.**

The card only stores a version number telling you a key exists. The actual key value lives on Blackboard's backend servers. Think of it like a lock that can verify a key is correct without ever holding a copy of it.

| Application | Key | Version | What This Tells Us |
|---|---|---|---|
| BBBBBB (Identity) | Key 0 | 02 | Has been rotated twice — actively managed |
| B3BBBB (Access) | Key 0 | 00 | Never rotated since provisioning |
| B3BBBB (Access) | Key 1 | 00 | Never rotated since provisioning |
| DABBBB (Transact) | Key 0 | 00 | Never rotated since provisioning |
| DABBBB (Transact) | Key 1 | 00 | Never rotated since provisioning |

The split key model used by the access and transactional apps is good security design. Door readers and dining terminals only hold Key 1 which can read but never write or modify. The university backend holds Key 0 which has full control.

## 5. How Transactions Actually Work

A common assumption is that your meal plan balance is stored on the card. It is not. Here is what actually happens when you tap at a dining hall:

Step 1 — You tap your card at the terminal
Step 2 — Terminal reads your card token using Key 1
Step 3 — Token is sent to Blackboard's server over the network
Step 4 — Server checks your balance and authorizes or declines
Step 5 — Terminal receives approval and processes the transaction
Step 6 — Balance updated in Blackboard's database

The card is a key, not a wallet. The 48-byte file in the transactional app contains an account token — an identifier that points to your account on the server. The balance number never touches the card.

## 6. Research Findings

### Finding 1 — Vendor Fingerprinting via AID Pattern

The Application IDs follow the pattern BB BB BB which is a known Blackboard Transact deployment signature. Any researcher with an NFC reader can identify the campus card vendor without any authentication simply by listing the application directory.

### Finding 2 — Plain Communication Mode on All Files

All three applications use Communication Settings 00 — plain mode. This means data is not MAC'd or encrypted at the transport layer during the RF transmission after authentication. More secure deployments use mode 01 (MAC'd) or 03 (fully encrypted). This is worth noting as a configuration observation.

### Finding 3 — Key Version Inconsistency

The Identity application (BBBBBB) shows Key 0 at version 02, indicating the key has been actively rotated. The Access Control and Transactional applications both show Key 0 and Key 1 at version 00, suggesting these keys have never been rotated since the cards were first provisioned. Inconsistent key rotation practices across applications is a noteworthy operational finding.

### Finding 4 — Open Application Directory

The PICC Free Directory List setting is true, meaning anyone can enumerate all application IDs on the card without authentication. This is by design in many deployments but contributes to the vendor fingerprinting finding above.

## 7. Tools and Methods Used

| Tool | Purpose | Method |
|---|---|---|
| Flipper Zero | Initial card capture | Passive unauthenticated NFC read |
| Flipper Zero CLI | Live APDU commands | GetVersion command via serial interface |
| libfreefare | Library reference | Source code analysis only |
| Python + pyserial | Flipper serial bridge | COM3 at 115200 baud |
| WSL2 + Ubuntu | Linux toolchain | Build environment for nfc-tools |

All testing was performed on the researcher's own card. No university infrastructure was probed, modified, or interacted with beyond passive RF observation.

## 8. What Is Protected vs. What Is Exposed

| Data | Accessible Without Auth? | Notes |
|---|---|---|
| Card UID | Yes | Always broadcast — by design |
| Application IDs | Yes | Free directory listing enabled |
| File metadata (sizes, types) | Yes | Standard DESFire behavior |
| Key version numbers | Yes | Version only — not key values |
| Hardware/software version | Yes | Confirmed via live APDU |
| File contents (identity data) | No | AES-128 — requires Key 0 |
| File contents (access credential) | No | AES-128 — requires Key 1 |
| File contents (meal token) | No | AES-128 — requires Key 1 |
| Balance information | No | Lives on server — not on card |